

QueryPilot

AI-Powered Business Intelligence Platform

Trello Tarzı Geliştirme Yol Haritası

BACKEND	DATABASE	ORM	ROADMAP
.NET	PostgreSQL	EF Core	48 Kart

Sırayla uygula | Kartı doğrudan geliştirme görevi olarak kullan | Ayrı Codex promptu yok

İçindekiler

Dokümanın büyük bölümü 48 geliştirme kartından oluşur. Her kartın tamamı doğrudan bir görev olarak kullanılabilir; ayrıca hazırlanmış coding promptları bulunmaz.

İçindekiler	2
1. QueryPilot Kısaca Nedir?	3
Projenin amacı	3
Dört parçanın görevi	3
Bir soru nasıl cevaplanır?	3
2. Sistem Nasıl Çalışıyor?	4
Sade mimari	4
Teknolojiler	4
3. Database Modeli ve Hardcode Kuralı	5
İki kritik finansal kural	5
Hardcode edilmeyecek business data	5
MVP sınırı	5
4. Ana Özellikler ve Kısa API Haritası	6
Projenin ana özellikleri	6
Özet API haritası	6
5. Geliştirme Fazları	7
Kartı nasıl kullanmalısınız?	7
6. Nice to Have Backlog	56
7. Proje Tamamlandığında	57
Ortaya çıkan ürün	57
CV için Türkçe örnek	57
English CV example	57

1. QueryPilot Kısaca Nedir?

Tek cümlelik açıklama

QueryPilot, PostgreSQL'deki satış, ürün, müşteri, sipariş ve iade kayıtlarını backend ile hesaplayan; AI'yı yalnız kullanıcı sorusunu anlamak ve gerçek sonucu açıklamak için kullanan bir Business Intelligence sistemidir.

Projenin amacı

- .NET Web API, PostgreSQL ve EF Core'u gerçek bir proje üzerinde öğrenmek.
- İlişkisel database, CRUD, validation, business logic ve REST API pratiği kazanmak.
- Revenue, trend, top products, category ve return analytics sorguları geliştirmek.
- AI ile backend arasındaki sorumluluk ayrımını doğru kurmak.
- Test, deployment ve README ile CV'de gösterilebilecek tamamlanmış ürün oluşturmak.

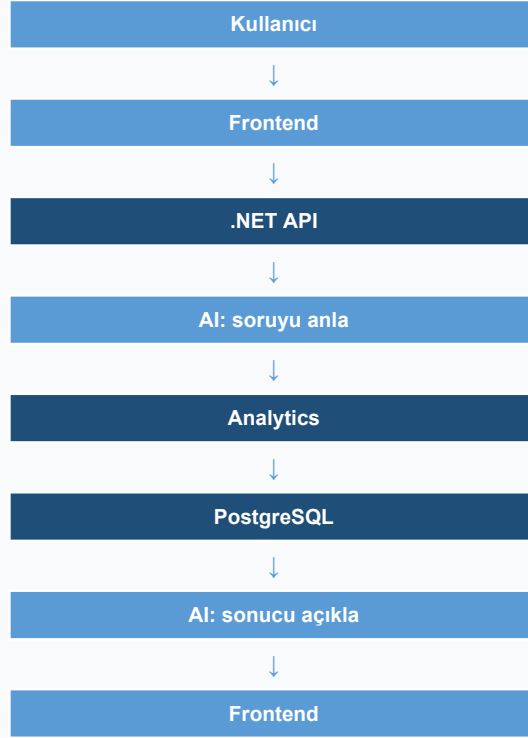
Dört parçanın görevi

Parça	Basit görevi	Kesin sınır
Database	Şirketin gerçek Product, Customer, Order, OrderItem, Category ve Return kayıtlarını tutar.	Hazır analytics veya AI cevabı tutmaz.
Backend	Veriyi okur/yazar, kuralları uygular, toplamaları ve analytics sonuçlarını hesaplar.	Client veya AI tarafından verilen business sayısını gerçek kabul etmez.
AI	Kullanıcı sorusunu anlar ve backend sonucunu insan dilinde açıklar.	Sayı uydurmaz, database yerine geçmez, raw SQL çalıştırmaz.
Frontend	Dashboard, grafikler, kayıt ekranları ve soru alanını kullanıcıya gösterir.	Finansal toplam veya güvenlik kararı hesaplamaz.

Bir soru nasıl cevaplanır?

- Kullanıcı 'Son üç ayda en çok satan 5 ürün ne?' diye sorar.
- AI bunun Top Products analizi, son üç ay ve 5 sonuç istediğini belirler.
- Backend AI çıktısını doğrular ve PostgreSQL'deki gerçek siparişleri sorgular.
- Backend ürün sıralamasını hesaplar; AI yalnız bu sonucu kısa bir dille açıklar.
- Frontend structured data ile liste veya grafik gösterebilir.

2. Sistem Nasıl Çalışıyor?



Değişmez kural

Database gerçeği tutar. Backend hesaplar ve karar verir. AI anlar ve açıklar. Frontend gösterir.

Sade mimari

```

QueryPilot.Api
  Features/
    Categories Products Customers Orders Returns Analytics Ai
  Data/
    AppDbContext Configurations Migrations Seed
  Common/
    Exceptions Validation Responses
  Configuration/
  Program.cs appsettings.json
  
```

Tek API projesi kullanılır. Microservices, Kafka, Event Bus, Event Sourcing, CQRS, MediatR, gereksiz Repository Pattern ve aşırı Clean Architecture eklenmez.

Teknolojiler

Alan	Karar	Neden?
Backend	.NET / ASP.NET Core Web API	REST API, Dependency Injection ve test desteği
Database	PostgreSQL	İlişkisel bütünlük ve güçlü aggregate sorguları
ORM	Entity Framework Core + Npgsql	LINQ, migration ve PostgreSQL bağlantısı
AI	Gemini veya OpenAI adapterı	Provider değişse bile ana mimari değişmez
Frontend	Daha sonra seçilecek	Backend bağımsız REST API olarak kalır

3. Database Modeli ve Hardcode Kuralı

Entity	Görevi	Önemli alanlar	İlişki
Category	Ürünleri gruplar	Id, Name, IsActive, CreatedAt	Bir Category'nin birçok Product'ı vardır
Product	Satılabilir ürünü ve güncel fiyatı tutar	Id, Name, SKU, CategoryId, UnitPrice, IsActive	Category'ye bağlı; OrderItemlarda kullanılır
Customer	Sipariş sahibini tutar	Id, Name, Email, City, IsActive	Bir Customer'ın birçok Order'ı vardır
Order	Sipariş başlığı ve hesaplanmış toplam	Id, CustomerId, OrderDate, Status, TotalAmount	Customer'a bağlı; birçok OrderItem içerir
OrderItem	Ürün, miktar ve satış anındaki fiyat	Id, OrderId, ProductId, Quantity, UnitPrice, LineTotal	Order ve Product'a bağlı; iadeleri olabilir
Return	Kısmi veya tam iade kaydı	Id, OrderItemId, Quantity, Reason, ReturnDate, Amount	Bir OrderItem'a bağlıdır

İki kritik finansal kural

- $OrderItem.LineTotal = Quantity \times \text{sipariş anındaki } UnitPrice$.
- $Order.TotalAmount = \text{bütün } OrderItem.LineTotal \text{ değerlerinin toplamı}$.
- Product fiyatı sonradan değişse bile eski OrderItem.UnitPrice değişmez.
- Return.Amount, güncel Product fiyatından değil OrderItem fiyat snapshotından hesaplanır.

Hardcode edilmeyecek business data

- Ürün, kategori ve müşteri bilgileri.
- Satış tutarı, sipariş sonucu ve iade değeri.
- En çok satan ürün, kategori performansı ve analytics sonuçları.
- AI'nın business cevabı, connection string, password ve API key.

```
// Yanlış
var bestProduct = "Laptop";
var monthlySales = 450000m;

// Doğru
var result = await analyticsService.GetTopProductsAsync(...);
```

DefaultPageSize, enum değerleri ve validation limitleri gibi teknik sabitler normaldir. Bir değer database değişince değişmesi gerekiyorsa business datadır ve kodda sabitlenmemelidir.

MVP sınırı

PostgreSQL, EF Core, altı ana entity, CRUD, order/return kuralları, beş analytics, AI soru anlama ve açıklama, belirsiz soru/hata yönetimi, testler, Swagger, CORS, deployment ve README tamamlanmadan Nice to Have işlerine geçilmez.

4. Ana Özellikler ve Kısa API Haritası

Projenin ana özellikleri

- Kategori, ürün ve müşteri kayıtlarını yönetme.
- Database fiyatını snapshot olarak transaction içinde sipariş oluşturma.
- Kısmi/tam iade ve satın alından fazla iadeyi engelleme.
- Sales Summary, Sales Trend, Top Products, Category Performance ve Return Analytics.
- Doğal dil sorusundan analiz ve parametre çıkarma.
- Belirsiz soruda kullanıcıdan açıklama isteme; provider hatasında güvenli fallback.
- Chart-ready JSON, Swagger, CORS, pagination ve standart hata cevapları.

Özet API haritası

Method	Endpoint	Ne yapıyor?
GET	/api/health	API ve database sağlık durumu
GET/POST/PUT	/api/categories	Kategori listeleme, ekleme ve güncelleme
GET/POST/PUT	/api/products	Ürün listeleme, arama, ekleme ve güncelleme
GET/POST/PUT	/api/customers	Müşteri listeleme, ekleme ve güncelleme
POST	/api/orders	Backend toplamıyla sipariş oluşturma
GET	/api/orders	Sayfalı ve filtreli sipariş listesi
GET	/api/orders/{id}	Sipariş ve item detayı
POST	/api/returns	Kısmi veya tam iade oluşturma
GET	/api/analytics/summary	Satış özeti
GET	/api/analytics/sales-trend	Günlük/haftalık/aylık trend
GET	/api/analytics/top-products	Adet veya gelire göre lider ürünler
GET	/api/analytics/categories	Kategori performansı
GET	/api/analytics/returns	İade analizi
POST	/api/ai/query	Doğal dil sorusundan gerçek BI cevabı

Frontend kararı

Bu endpointler çalıştığında React, Vue, Angular veya başka bir frontend aynı REST sözleşmesini kullanabilir. Bu dokümanda frontend kodu yazılmayacaktır.

5. Geliştirme Fazları

Roadmap 48 ana karttan oluşur. Küçük DTO, validator veya endpoint işleri ayrı kartlara bölünmez; ilgili feature kartının checklistinde tamamlanır. Her kartı sırayla bitirin.

Faz	Adımlar	Bu fazda ne yapıyoruz?	Geçiş şartı
FAZ 1 - Projeyi Başlatma	Adım 1-5	Çalışan bir .NET API, güvenli configuration, PostgreSQL bağlantısı ve health kontrolü hazırla.	API derleniyor ve database bağlantısı kontrol edilebiliyor.
FAZ 2 - Database Modeli	Adım 6-10	Şirket verisini tutacak entityleri ve ilişkileri kur; migration ve demo verisini hazırla.	Fresh PostgreSQL veritabanı tek migration akışıyla kurulabiliyor.
FAZ 3 - Temel Backend Modülleri	Adım 11-14	Ortak API kurallarını ve Category, Product, Customer işlemlerini tamamlama.	Temel kayıtlar Swagger üzerinden güvenli şekilde yönetilebiliyor.
FAZ 4 - Sipariş Sistemi	Adım 15-19	Fiyatı database'den alan, toplamı backend'de hesaplayan transactionlı sipariş akışını kur.	Sipariş oluşturma, listeleme ve detay görüntüleme finansal olarak doğru.
FAZ 5 - İade Sistemi	Adım 20-22	Kısmi/tam iadeyi ve satın alımdan fazla iade engelini uygula.	İade miktarı ve tutarı backend kurallarıyla güvenli hesaplanıyor.
FAZ 6 - Analytics	Adım 23-30	Beş ana BI analizini gerçek PostgreSQL kayıtlarından hesapla.	AI olmadan tüm analytics endpointleri doğru ve dinamik sonuç üretiyor.
FAZ 7 - AI Entegrasyonu	Adım 31-36	Doğal dil sorusunu yapılandırılmış analize çevir; belirsizlik ve provider hatalarını yönet.	AI soruyu anlıyor fakat business sayılarını üretmiyor.
FAZ 8 - AI ve Analytics'i Birleştirme	Adım 37-39	AI sonucunu doğruyla, doğru analytics fonksiyonuna yönlendir ve gerçek sonucu açıkla.	Tek bir soru endpointi uçtan uca çalışıyor.
FAZ 9 - Testler	Adım 40-42	Sipariş, iade, analytics, AI ve HTTP davranışlarını otomatik testlerle kanıtla.	Kritik kurallar temiz test veritabanında tekrar üretilebiliyor.
FAZ 10 - Dinamik Çalışma Kanıtı	Adım 43	Yeni sipariş ekleyerek sonuçların database değişimine göre güncellendiğini göster.	Hardcode kullanılmadığı canlı olarak kanıtlanıyor.
FAZ 11 - Frontend İçin Hazırlık	Adım 44-46	Frontend seçmeden API sözleşmesini, CORS'u, Swagger'ı ve README'yi tamamlama.	React, Vue veya Angular aynı REST API'yi kullanabilir.
FAZ 12 - Deployment	Adım 47-48	Cloud PostgreSQL ve production API'yi secretsız, testli ve gözlemlenebilir şekilde yayınla.	HTTPS ortamında health, analytics ve AI smoke testleri geçiyor.

Kartı nasıl kullanmalısınız?

- Kartın 'Yapılacaklar' listesini tamamlayın.
- Dikkat edilmesi gerekenleri code review kontrolü gibi kullanın.
- 'Bu adım bittiğinde' sonuçları gerçekten çalışmadan sonraki karta geçmeyin.
- Kartın tamamı doğrudan Codex'e görev olarak verilebilir; ayrı prompt bulunmaz.
- Her kart sonunda build/test kontrolü yapıp küçük ve anlamlı Git commit oluşturun.

ADIM 1 - Git Repository, Solution ve Web API'yi Başlat

FAZ	Faz 1 - Projeyi Başlatma	ÖN KOŞUL	Yok
-----	--------------------------	----------	-----

Bu fazın hedefi

Önce çalışan ve güvenli bir geliştirme ortamı kur. Business feature yazmaya başlamadan build, database ve health zincirini doğrula.

Bu adımda ne yapıyoruz?

Boş klasörü çalışan ve Git ile takip edilen bir .NET 8 Web API projesine dönüştürüyoruz.

Yapılacaklar

- [] Git repository oluştur ve ana branch'i kontrol et.
- [] .NET 8 solution oluştur.
- [] src altında QueryPilot.Api Web API projesini oluştur.
- [] API projesini solution'a ekle.
- [] Nullable ve implicit usings ayarlarını açık tut.
- [] Template içindeki kullanılmayacak örnek endpointleri kaldır.
- [] dotnet restore ve dotnet build komutlarını çalıştır.
- [] API'yi ayağa kaldır ve tarayıcıdan erişildiğini doğrula.
- [] İlk anlamlı Git commitini oluştur.

Dikkat edilmesi gerekenler

- Gereksiz NuGet paketi ekleme.
- Build hatası varken adımı tamamlandı sayma.

Bu adım bittiğinde

- ☐ Solution açılıyor.
- ☐ API sıfır hata ile derleniyor.
- ☐ Git geçmişinde ilk commit görünüyor.

Bu adımda öğrenilecekler

Git • .NET solution • Web API • Build süreci

ADIM 2 - Sade Feature Klasör Yapısını Kur

FAZ

Faz 1 - Projeyi Başlatma

ÖN KOŞUL

Adım 1

Bu adımda ne yapıyoruz?

Kod büyüdüğünde kaybolmamak için feature bazlı, tek proje içinde anlaşılır bir klasör düzeni kuruyoruz.

Yapılacaklar

- [] Features klasörünü oluştur.
- [] Categories, Products, Customers, Orders, Returns, Analytics ve Ai klasörlerini ekle.
- [] Data altında AppDbContext, Configurations, Migrations ve Seed bölümlerini planla.
- [] Common altında Exceptions, Validation ve Responses klasörlerini oluştur.
- [] Configuration klasörünü ekle.
- [] Namespace'lerin klasör yapısıyla uyumlu olmasını sağla.
- [] Klasör ağacını README taslağına ekle.
- [] Projenin hâlâ derlendiğini doğrula.

Dikkat edilmesi gerekenler

- Microservice, CQRS, MediatR veya çok katmanlı solution ekleme.
- Bütün feature kodlarını tek klasöre toplama.

Bu adım bittiğinde

- ☐ Her ana özellik için açık bir klasör var.
- ☐ Tek deploy edilen API projesi korunuyor.

Bu adımda öğrenilecekler

Modüler monolith • Feature organizasyonu • Namespace

ADIM 3 - Configuration ve Secret Yönetimini Hazırla

FAZ	Faz 1 - Projeyi Başlatma	ÖN KOŞUL	Adım 1-2
-----	--------------------------	----------	----------

Bu adımda ne yapıyoruz?

Database ve AI ayarlarını source code'a yazmadan farklı ortamlardan okuyabilecek yapı kuruyoruz.

Yapılacaklar

- [] appsettings.json içinde ConnectionStrings, AI ve CORS için boş/güvenli bölümler oluştur.
- [] Development ayarlarını appsettings.Development.json içinde tut.
- [] AI provider, model adı ve timeout için options modeli hazırla.
- [] CORS originlerini configuration üzerinden okunacak şekilde planla.
- [] dotnet user-secrets özelliğini API projesinde başlat.
- [] Yerel database connection stringini user-secrets içine ekle.
- [] AI API key için user-secrets komutunu hazırla; henüz provider yoksa gerçek çağrı yapma.
- [] .gitignore içinde bin, obj, IDE ve secret dosyalarının dışlandığını doğrula.

Dikkat edilmesi gerekenler

- Connection string, password veya API key'i appsettings dosyasına yazma.
- Secret değerlerini log veya ekran görüntüsünde paylaşma.

Bu adım bittiğinde

- ☐ Uygulama configuration bölümlerini okuyabiliyor.
- ☐ Gerçek secret Git durumunda görünmüyor.

Bu adımda öğrenilecekler

Configuration • Options pattern • User Secrets • .gitignore

ADIM 4 - PostgreSQL ve EF Core Bağlantısını Kur

FAZ Faz 1 - Projeyi Başlatma ÖN KOŞUL Adım 3

Bu adımda ne yapıyoruz?

API'nin gerçek PostgreSQL veritabanına EF Core üzerinden bağlanmasını sağlıyoruz.

Yapılacaklar

- [] Yerel PostgreSQL kur veya güvenli bir development instance hazırla.
- [] querypilot_dev isimli database oluştur.
- [] Uygulama için ayrı bir database kullanıcısı oluştur.
- [] Npgsql.EntityFrameworkCore.PostgreSQL paketini ekle.
- [] EF Core design paketini migration işlemleri için ekle.
- [] DbContext bağlantısını Program.cs içinde Dependency Injection ile kaydet.
- [] Connection stringin yalnız configuration üzerinden geldiğini kontrol et.
- [] Basit bir bağlantı testi yap ve PostgreSQL sürümünü doğrula.

Dikkat edilmesi gerekenler

- Uygulamada PostgreSQL superuser kullanma.
- Connection stringi C# dosyasına yazma.

Bu adım bittiğinde

- ☐ API DbContext'i Dependency Injection üzerinden oluşturabiliyor.
- ☐ Geçerli ayarla PostgreSQL'e bağlantı kuruluyor.

Bu adımda öğrenilecekler

PostgreSQL • EF Core provider • Dependency Injection • Connection string

ADIM 5 - ApplicationDbContext ve Health Check'i Tamamla

FAZ

Faz 1 - Projeyi Başlatma

ÖN KOŞUL

Adım 4

Bu adımda ne yapıyoruz?

Veri erişiminin merkezini ve API/database sağlık kontrolünü kuruyoruz.

Yapılacaklar

- [] Data/AppDbContext.cs dosyasını oluştur.
- [] DbContext options constructorını ekle.
- [] Entity configurationlarının assembly içinden otomatik uygulanmasını planla.
- [] Health check servislerini Program.cs içine ekle.
- [] Database bağlantısını da kontrol eden health check ekle.
- [] GET /api/health endpointini oluştur.
- [] Database çalışırken endpointin başarılı cevap verdiğini doğrula.
- [] Database kapalıyken endpointin kontrollü şekilde başarısız olduğunu doğrula.
- [] Health response içinde connection string veya password görünmediğini kontrol et.

Dikkat edilmesi gerekenler

- DbContext içine business logic yazma.
- Health response'ta altyapı secretlarını gösterme.

Bu adım bittiğinde

- ☐ ApplicationDbContext çalışıyor.
- ☐ /api/health API ve database durumunu gösterebiliyor.

Bu adımda öğrenilecekler

DbContext • Health check • Dependency readiness

ADIM 6 - Database Modelini ve İlişkileri Planla

FAZ

Faz 2 - Database Modeli

ÖN KOŞUL

Adım 5

Bu fazın hedefi

Veri modelini bir kez doğru kur. Sonraki bütün CRUD, sipariş ve analytics işleri bu ilişkileri kullanacak.

Bu adımda ne yapıyoruz?

Kod yazmadan önce hangi verinin nerede tutulacağını ve tabloların nasıl bağlanacağını netleştiriyoruz.

Yapılacaklar

- [] Category, Product, Customer, Order, OrderItem ve Return entitylerini listele.
- [] Her entity için zorunlu alanları belirle.
- [] Category ile Product arasındaki bire-çok ilişkii çiz.
- [] Customer ile Order arasındaki bire-çok ilişkii çiz.
- [] Order ile OrderItem arasındaki bire-çok ilişkii çiz.
- [] Product ile OrderItem ve OrderItem ile Return ilişkilerini çiz.
- [] Para alanlarında decimal kullanma kararını yaz.
- [] Bütün tarihlerin UTC tutulacağını belirle.
- [] Finansal kayıtların hard delete edilmeyeceğini kararlaştır.

Dikkat edilmesi gerekenler

- Analytics sonuçlarını ayrı tabloya hazır değer olarak koyma.
- Entity ilişkilerini sadece ezberle değil, gerçek kullanım akışına göre kur.

Bu adım bittiğinde

- ☐ Entity ve ilişki haritası nettir.
- ☐ Silme, para ve tarih kuralları yazılıdır.

Bu adımda öğrenilecekler

Primary key • Foreign key • One-to-many • Decimal • UTC

ADIM 7 - Category, Product ve Customer Entitylerini Oluştur

FAZ

Faz 2 - Database Modeli

ÖN KOŞUL

Adım 6

Bu adımda ne yapıyoruz?

Ürün kataloğu ve müşteri kayıtlarının PostgreSQL'de tutulacağı temel entityleri oluşturuyoruz.

Yapılacaklar

- [] Category entitysine Id, Name, IsActive ve CreatedAt alanlarını ekle.
- [] Product entitysine Id, Name, SKU, CategoryId, UnitPrice, IsActive ve CreatedAt alanlarını ekle.
- [] Customer entitysine Id, Name, Email, City, IsActive ve CreatedAt alanlarını ekle.
- [] Product ile Category navigation ilişkisini kur.
- [] Category Name ve Product SKU için unique kural planla.
- [] Customer email adresini normalize ederek unique kontrolüne hazırla.
- [] Product UnitPrice için decimal kullan.
- [] İsim, SKU ve email alanlarına makul uzunluk sınırları belirle.
- [] Entityleri ApplicationDbContext içine DbSet olarak ekle.

Dikkat edilmesi gerekenler

- Ürün, müşteri veya kategori örneklerini entity koduna yazma.
- Fiyat için double/float kullanma.

Bu adım bittiğinde

- ☐ Üç entity derleniyor.
- ☐ Product bir Category'ye bağlanabiliyor.
- ☐ DbContext üç tabloyu modelleyebiliyor.

Bu adımda öğrenilecekler

Entity • Unique constraint • Navigation property • DbSet

ADIM 8 - Order ve OrderItem Entitylerini Oluştur

FAZ

Faz 2 - Database Modeli

ÖN KOŞUL

Adım 6-7

Bu adımda ne yapıyoruz?

Sipariş başlığını, sipariş satırlarını ve satış anındaki fiyatı saklayacak modeli oluşturuyoruz.

Yapılacaklar

- [] Order entitysine Id, CustomerId, OrderDate, Status, TotalAmount ve CreatedAt alanlarını ekle.
- [] OrderStatus için anlaşılır enum değerleri oluştur.
- [] OrderItem entitysine Id, OrderId, ProductId, Quantity, UnitPrice ve LineTotal alanlarını ekle.
- [] Order ile Customer ilişkisini kur.
- [] Order ile OrderItem ilişkisini kur.
- [] OrderItem ile Product ilişkisini kur.
- [] OrderItem.UnitPrice alanının sipariş anındaki fiyatı saklayacağını yorum veya dokümanla açıkla.
- [] LineTotal ve TotalAmount alanlarında decimal kullan.
- [] Order ve OrderItem DbSetlerini ApplicationDbContext'e ekle.

Dikkat edilmesi gerekenler

- TotalAmount ve LineTotal değerlerini frontend'den alınacak alan gibi tasarlama.
- Product fiyatı değişince eski OrderItem fiyatı değişmemelidir.

Bu adım bittiğinde

- ☐ Sipariş modeli müşteri ve ürünlerle ilişkilidir.
- ☐ Fiyat snapshotı için gerekli alanlar hazırdır.

Bu adımda öğrenilecekler

Aggregate • Enum • Price snapshot • Finansal model

ADIM 9 - Return Entitysini ve Database Kurallarını Oluştur

FAZ

Faz 2 - Database Modeli

ÖN KOŞUL

Adım 8

Bu adımda ne yapıyoruz?

İade işlemlerini sipariş satırına bağlayıp database seviyesindeki temel kuralları belirliyoruz.

Yapılacaklar

- [] Return entitysine Id, OrderItemId, Quantity, Reason, ReturnDate ve Amount alanlarını ekle.
- [] Return ile OrderItem ilişkisini kur.
- [] Return entitysini ApplicationDbContext'e ekle.
- [] Her entity için Fluent API configuration dosyası oluştur.
- [] Zorunlu alanları, string uzunluklarını ve decimal precision değerlerini tanımla.
- [] Category Name, Product SKU ve normalized Email için unique constraint ekle.
- [] Foreign key kolonlarına gerekli indexleri ekle.
- [] OrderDate ve ReturnDate için analytics sorgularını destekleyecek indexleri değerlendir.
- [] Finansal kayıtların cascade delete ile kaybolmasını engelle.

Dikkat edilmesi gerekenler

- Return Amount değerini kullanıcıdan alınacak şekilde tasarlama.
- Order/OrderItem/Return kayıtlarını kolayca hard delete etme.

Bu adım bittiğinde

- ☐ Altı ana entity configuration ile tamamdır.
- ☐ Unique ve foreign key kuralları migrationa hazırdır.

Bu adımda öğrenilecekler

Fluent API • Index • Precision • Delete behavior

ADIM 10 - Migration ve Gerçekçi Demo Verisini Hazırla

FAZ

Faz 2 - Database Modeli

ÖN KOŞUL

Adım 7-9

Bu adımda ne yapıyoruz?

Kod modelini PostgreSQL şemasına çevirip analytics için gerçekçi sahte kayıtlar oluşturuyoruz.

Yapılacaklar

- [] InitialCreate migrationını oluştur.
- [] Migration içindeki tabloları, foreign keyleri ve unique kuralları gözden geçir.
- [] Migrationı querpilot_dev database'ine uygula.
- [] Database tablolarını bir yönetim aracı veya SQL ile kontrol et.
- [] Yaklaşık 10 kategori, 100 ürün ve 500 müşteri üretecek seed planı oluştur.
- [] Siparişleri birkaç aylık veya yıllık tarih aralığına dağıt.
- [] Farklı ürün, kategori, müşteri ve şehir dağılımları oluştur.
- [] Sipariş satırlarını ve iadeleri temel kayıtlardan üret.
- [] Seed işlemini tekrar çalıştırınca duplicate veri oluşmasını engelle.
- [] Production ortamında demo seed çalışmasını kapat.

Dikkat edilmesi gerekenler

- Aylık satış toplamı veya en çok satan ürün sonucunu seed içine hazır yazma.
- Seed verisini gerçek kişi bilgileriyle oluşturma.

Bu adım bittiğinde

- ☐ Fresh database migration ile kuruluyor.
- ☐ Seed analytics için anlamlı fakat sahte veri üretiyor.

Bu adımda öğrenilecekler

Migration • Seed data • Veri dağılımı • Idempotency

ADIM 11 - Ortak Validation, Error ve Pagination Altyapısını Kur

FAZ

Faz 3 - Temel Backend Modülleri

ÖN KOŞUL

Adım 10

Bu fazın hedefi

Feature'lar için ortak API davranışını kur, ardından temel kayıt modüllerini tamamla.

Bu adımda ne yapıyoruz?

Her feature'ın aynı hata ve listeleme davranışını kullanacağı ortak API temelini kuruyoruz.

Yapılacaklar

- [] Validation hataları için standart response biçimi oluştur.
- [] Bulunamayan kayıt, conflict ve beklenmeyen hata için ortak exception türleri tanımla.
- [] Global exception handling ekle.
- [] 400, 404, 409, 500 ve AI için 503 durumlarını ProblemDetails ile döndür.
- [] Response içine takip için traceId ekle.
- [] Liste endpointleri için ortak pagination modeli oluştur.
- [] Varsayılan page size ve maksimum page size belirle.
- [] Beklenmeyen hataları logla.
- [] API key, password, email ve hassas request içeriğini loglama.
- [] Her endpoint içinde tekrar tekrar try/catch yazılmadığını kontrol et.

Dikkat edilmesi gerekenler

- Stack trace'i production response içinde gösterme.
- Validationı sadece frontend'e bırakma.

Bu adım bittiğinde

- ☐ API hataları aynı formatta dönüyor.
- ☐ Pagination sınırları ortak çalışıyor.
- ☐ Secret içermeyen loglar oluşuyor.

Bu adımda öğrenilecekler

ProblemDetails • Global exception handling • Pagination • Logging

ADIM 12 - Category Modülünü Tamamla

FAZ

Faz 3 - Temel Backend Modülleri

ÖN KOŞUL

Adım 11

Bu adımda ne yapıyoruz?

Kategorileri backend üzerinden eklenebilir, listelenebilir ve güncellenebilir hale getiriyoruz.

Yapılacaklar

- [] Category create, update, list ve response DTO'larını oluştur.
- [] Kategori adının boş veya aşırı uzun olmasını engelle.
- [] Aynı kategori adının tekrar eklenmesini engelle.
- [] CategoryService içinde database işlemlerini yaz.
- [] Kategori listeleme endpointine pagination ve aktiflik filtresi ekle.
- [] Yeni kategori ekleyen POST endpointini oluştur.
- [] Kategori güncelleyen PUT endpointini oluştur.
- [] Hard delete yerine IsActive ile pasifleştirme kullan.
- [] Entity yerine DTO döndür.
- [] Swagger üzerinden create, duplicate, update ve list testlerini yap.
- [] Kaydın PostgreSQL'de oluştuğunu doğrula.

Dikkat edilmesi gerekenler

- Kategori adlarını kod içinde sabitleme.
- Endpoint içine business logic doldurma.

Bu adım bittiğinde

- ☐ Kategori eklenebiliyor, listelenebiliyor ve güncellenebiliyor.
- ☐ Duplicate kategori engelleniyor.

Bu adımda öğrenilecekler

CRUD • DTO • Service • Validation • REST

ADIM 13 - Product Modülünü Tamamla

FAZ

Faz 3 - Temel Backend Modülleri

ÖN KOŞUL

Adım 12 ve Category hazır

Bu adımda ne yapıyoruz?

Ürünlerin PostgreSQL'de tutulmasını ve backend üzerinden güvenli şekilde yönetilmesini sağlıyoruz.

Yapılacaklar

- [] Product create, update, list ve response DTO'larını oluştur.
- [] Name, SKU, CategoryId, UnitPrice ve IsActive alanlarını validate et.
- [] UnitPrice değerinin sıfırdan büyük olmasını zorunlu tut.
- [] Aynı SKU'nun ikinci kez kullanılmasını engelle.
- [] Kategori database'de yoksa ürün oluşturma.
- [] ProductService içinde create, update ve list işlemlerini yaz.
- [] Ürün ekleme ve güncelleme endpointlerini oluştur.
- [] Liste endpointine pagination, aktiflik ve kategori filtresi ekle.
- [] İsim veya SKU ile arama ekle.
- [] Ürünü silmek yerine pasifleştirme davranışını uygula.
- [] Response içinde kategori özetini DTO ile göster.
- [] Swagger ve PostgreSQL üzerinden positive/negative testleri yap.

Dikkat edilmesi gerekenler

- Ürün fiyatını veya isimlerini kod içine yazma.
- Entityyi doğrudan API response olarak döndürme.
- Geçmiş OrderItem fiyatlarını product update ile değiştirme.

Bu adım bittiğinde

- ☐ Ürün ekleme, listeleme, arama, filtreleme ve güncelleme çalışıyor.
- ☐ Invalid kategori, fiyat ve duplicate SKU engelleniyor.

Bu adımda öğrenilecekler

Foreign key • Projection • Search • Filtering • EF Core

ADIM 14 - Customer Modülünü Tamamla

FAZ

Faz 3 - Temel Backend Modülleri

ÖN KOŞUL

Adım 11

Bu adımda ne yapıyoruz?

Sipariş verecek müşterileri güvenli ve filtrelenebilir şekilde yönetiyoruz.

Yapılacaklar

- [] Customer create, update, list ve response DTO'larını oluştur.
- [] Name, Email, City ve IsActive alanlarını validate et.
- [] Email formatını kontrol et.
- [] Emaili normalize et ve duplicate email politikasını uygula.
- [] CustomerService içinde database işlemlerini yaz.
- [] Müşteri ekleme ve güncelleme endpointlerini oluştur.
- [] Liste endpointine pagination, arama, şehir ve aktiflik filtresi ekle.
- [] Sipariş geçmişli olan müşteriyi silmek yerine pasifleştir.
- [] API response içinde gereksiz kişisel veri döndürme.
- [] Email adresinin loglara yazılmadığını kontrol et.
- [] Swagger üzerinden valid/invalid ve duplicate email testleri yap.

Dikkat edilmesi gerekenler

- Gerçek müşteri bilgilerini demo verisi olarak kullanma.
- PII içeren requestleri tam olarak loglama.

Bu adım bittiğinde

- ☐ Müşteri CRUD ve filtreler çalışıyor.
- ☐ Geçersiz/duplicate email engelleniyor.

Bu adımda öğrenilecekler

PII • Normalization • CRUD • Filtering

ADIM 15 - Sipariş Request ve Response Modellerini Oluştur

FAZ

Faz 4 - Sipariş Sistemi

ÖN KOŞUL

Product ve Customer modülleri hazır

Bu fazın hedefi

Sipariş fiyatını ve toplamını yalnız backend belirlesin; kayıt işlemi atomik olsun.

Bu adımda ne yapıyoruz?

Frontend'in yalnız müşteri, ürün ve miktar göndereceği güvenli sipariş sözleşmesini hazırlıyoruz.

Yapılacaklar

- [] CreateOrderRequest içine CustomerId ve item listesini ekle.
- [] Her item için yalnız ProductId ve Quantity kabul et.
- [] Request içinde UnitPrice, LineTotal veya TotalAmount alanı bulundurma.
- [] En az bir sipariş satırı zorunlu tut.
- [] Quantity değerinin sıfırdan büyük olmasını zorunlu tut.
- [] Aynı productın request içinde iki kez gelmesi durumunda birleştirme veya hata politikasını belirle.
- [] Order list response için hafif DTO oluştur.
- [] Order detail response için item ve müşteri özetini içeren DTO oluştur.
- [] OrderDate alanının backend tarafından atanma politikasını belirle.
- [] Validation hatalarını ortak error formatında döndürmeye hazırla.

Dikkat edilmesi gerekenler

- Frontend'in toplam veya fiyat belirlemesine izin verme.
- Entity graphını doğrudan serialize etme.

Bu adım bittiğinde

- ☐ Sipariş requesti yalnız güvenli alanları kabul ediyor.
- ☐ Liste ve detay response modelleri ayırdır.

Bu adımda öğrenilecekler

DTO • Over-posting • Request validation • API contract

ADIM 16 - Müşteri ve Ürünleri Database'den Doğrula

FAZ

Faz 4 - Sipariş Sistemi

ÖN KOŞUL

Adım 15

Bu adımda ne yapıyoruz?

Sipariş kaydetmeden önce müşteri ve tüm ürünlerin gerçekten database'de ve aktif olduğunu kontrol ediyoruz.

Yapılacaklar

- [] CustomerId ile müşteriye database'den bul.
- [] Müşteri yoksa sipariş işlemini durdur ve anlaşılır hata döndür.
- [] Pasif müşterinin yeni sipariş vermesini engelle.
- [] Requestteki tüm ProductId değerlerini tek sorguda database'den getir.
- [] Eksik ürün varsa hangi kaydın bulunamadığını güvenli şekilde bildir.
- [] Pasif ürünlerin siparişe eklenmesini engelle.
- [] Her ürünün güncel UnitPrice değerini database kaydından al.
- [] Database'den gelen fiyatı yeni OrderItem.UnitPrice alanına kopyala.
- [] Aynı siparişte çok sayıda item olduğunda N+1 sorgu oluşmadığını kontrol et.
- [] Herhangi bir invalid item varsa henüz hiçbir sipariş kaydetme.

Dikkat edilmesi gerekenler

- Kullanıcının gönderdiği fiyatı kullanma.
- Bazı itemları kaydedip bazılarını atlama.

Bu adım bittiğinde

- ☐ Müşteri ve ürün kontrolü tamamlanmadan kayıt yapılmıyor.
- ☐ OrderItem fiyatı database fiyatından snapshot alıyor.

Bu adımda öğrenilecekler

Bulk query • Business validation • Price snapshot • N+1

ADIM 17 - Sipariş Toplamlarını Hesapla ve Transaction İçinde Kaydet

FAZ

Faz 4 - Sipariş Sistemi

ÖN KOŞUL

Adım 16

Bu adımda ne yapıyoruz?

Sipariş satırlarını ve toplamı backend'de hesaplayıp tek transaction içinde PostgreSQL'e kaydediyoruz.

Yapılacaklar

- [] Her OrderItem için LineTotal = Quantity x UnitPrice hesapla.
- [] Order.TotalAmount değerini bütün LineTotal değerlerinin toplamından üret.
- [] Para hesaplarında decimal kullan.
- [] Order ve OrderItem kayıtlarını tek aggregate olarak oluştur.
- [] Bütün kayıt işlemini transaction içinde çalıştır.
- [] Herhangi bir hata olursa transactionı geri al.
- [] POST /api/orders endpointini oluştur.
- [] Başarılı işlemde 201 Created ve order detail DTO döndür.
- [] Location veya detail adresini response içinde belirt.
- [] Kaydedilen TotalAmount ile item toplamlarının eşit olduğunu doğrula.
- [] Product fiyatını sonradan değiştir ve eski OrderItem.UnitPrice değerinin aynı kaldığını kontrol et.
- [] Sipariş oluşturma olayını secretsız logla.

Dikkat edilmesi gerekenler

- TotalAmount değerini requestten alma.
- float/double ile finansal hesap yapma.
- Transaction dışında yarım sipariş bırakma.

Bu adım bittiğinde

- ☐ Geçerli sipariş tek işlemde kaydoluyor.
- ☐ Toplamlar backend tarafından doğru hesaplanıyor.
- ☐ Price snapshot korunuyor.

Bu adımda öğrenilecekler

Transaction • Decimal calculation • Aggregate • 201 Created

ADIM 18 - Sipariş Listeleme ve Detay Endpointlerini Oluştur

FAZ

Faz 4 - Sipariş Sistemi

ÖN KOŞUL

Adım 17

Bu adımda ne yapıyoruz?

Kaydedilen siparişleri sayfalı olarak listeleyip tek siparişin ayrıntısını görüntülüyoruz.

Yapılacaklar

- [] GET /api/orders endpointini oluştur.
- [] Listelemeye pagination ekle.
- [] Tarih aralığı, status ve customer filtresi ekle.
- [] Sonuçları OrderDate ve Id ile kararlı sırala.
- [] Liste için yalnız gerekli alanları projection ile getir.
- [] GET /api/orders/{id} endpointini oluştur.
- [] Detay response içinde customer özeti, itemlar, fiyat snapshotı ve toplamaları göster.
- [] Varsa iade özetini detail response'a ekle.
- [] Olmayan order için 404 ProblemDetails döndür.
- [] from tarihi to tarihinden büyükse 400 döndür.
- [] Entity navigationlarını kontrolsüz şekilde serialize etme.
- [] Swagger üzerinden filtre ve 404 testlerini yap.

Dikkat edilmesi gerekenler

- Liste endpointinde bütün entity graphını Include ile çekme.
- Sınırsız sonuç döndürme.

Bu adım bittiğinde

- ☐ Siparişler filtrelenebilir ve sayfalıdır.
- ☐ Tek siparişin finansal detayı doğru görünür.

Bu adımda öğrenilecekler

Projection • Pagination • Filtering • 404

ADIM 19 - Sipariş Sistemini Uçtan Uca Doğrula

FAZ

Faz 4 - Sipariş Sistemi

ÖN KOŞUL

Adım 15-18

Bu adımda ne yapıyoruz?

Sipariş kurallarının gerçek HTTP istekleri ve PostgreSQL kayıtlarıyla doğru çalıştığını kanıtlıyoruz.

Yapılacaklar

- [] Geçerli müşteri ve ürünlerle tek itemlı sipariş oluştur.
- [] Birden fazla ürün içeren sipariş oluştur.
- [] Olmayan müşteriyle sipariş oluşturmayı dene.
- [] Pasif müşteriyle sipariş oluşturmayı dene.
- [] Olmayan ve pasif ürünlerle sipariş oluşturmayı dene.
- [] Quantity değerini 0 ve negatif gönder.
- [] Requeste fazladan fiyat/toplam alanı gönderildiğinde kullanılmadığını doğrula.
- [] Product fiyatını değiştir ve eski sipariş snapshotının değişmediğini doğrula.
- [] Item toplamları ile Order.TotalAmount değerini karşılaştır.
- [] Hatalı item olduğunda database'de yarım order kalmadığını doğrula.
- [] Liste ve detay endpointlerinden yeni siparişi kontrol et.

Dikkat edilmesi gerekenler

- Yalnız happy path test edip edge caseleri atlama.
- Beklenen toplamı production koduna sabit sayı olarak ekleme.

Bu adım bittiğinde

- ☐ Sipariş happy path ve bütün kritik negative testlerden geçiyor.
- ☐ Database kayıtları finansal olarak tutarlı.

Bu adımda öğrenilecekler

Uçtan uca test • Transaction doğrulama • Edge case

ADIM 20 - İade Requestini ve İş Kurallarını Tasarla

FAZ

Faz 5 - İade Sistemi

ÖN KOŞUL

Sipariş sistemi hazır

Bu fazın hedefi

Kısmi iadeyi destekle ve toplam iadenin satın alınanı aşmasını engelle.

Bu adımda ne yapıyoruz?

Kullanıcının bir sipariş satırındaki ürünlerin tamamını veya bir kısmını güvenli biçimde iade edebilmesini tasarlıyoruz.

Yapılacaklar

- [] CreateReturnRequest içine OrderItemId, Quantity ve Reason alanlarını ekle.
- [] Quantity değerinin sıfırdan büyük olmasını zorunlu tut.
- [] Reason alanı için boşluk ve uzunluk kontrolü ekle.
- [] OrderItem kaydının bulunmasını zorunlu tut.
- [] Satın alınan toplam miktarı OrderItem.Quantity üzerinden oku.
- [] Aynı OrderItem için daha önce yapılan iadelerin Quantity toplamını hesapla.
- [] Kalan iade edilebilir miktarı belirle.
- [] Yeni iade ile toplam iadenin satın alınan miktarı aşmasını engelle.
- [] İade için uygun order status politikasını belirle.
- [] Return.Amount değerinin OrderItem.UnitPrice x Quantity olacağını belirle.

Dikkat edilmesi gerekenler

- Güncel Product.UnitPrice ile iade tutarı hesaplama.
- Önceki iadeleri hesaba katmadan yeni iade kabul etme.

Bu adım bittiğinde

- ☐ Kısmi ve tam iade kuralları nettir.
- ☐ Over-return backend tarafından engellenecektir.

Bu adımda öğrenilecekler

Cumulative validation • Partial return • Snapshot price

ADIM 21 - İade Servisini ve Endpointini Oluştur

FAZ

Faz 5 - İade Sistemi

ÖN KOŞUL

Adım 20

Bu adımda ne yapıyoruz?

İade kurallarını uygulayıp Return kaydını transaction içinde PostgreSQL'e yazıyoruz.

Yapılacaklar

- [] ReturnService oluştur.
- [] OrderItem ve önceki iadeleri database'den yükle.
- [] Kalan iade miktarını tekrar backend içinde doğrula.
- [] Return.Amount değerini fiyat snapshotından hesapla.
- [] ReturnDate değerini UTC olarak backend'de oluştur.
- [] Return kaydını transaction içinde kaydet.
- [] POST /api/returns endpointini oluştur.
- [] Başarılı işlemde 201 ve ReturnResponse döndür.
- [] Satın alınandan fazla iade durumunda 409 veya belirlenen ortak conflict cevabını döndür.
- [] Aynı anda gelen iki iade isteğinin over-return oluşturma riskini transaction/concurrency yaklaşımıyla ele al.
- [] İade olayını hassas veri içermeden logla.

Dikkat edilmesi gerekenler

- İade tutarını requestten alma.
- Finansal Return kaydını hard delete etme.

Bu adım bittiğinde

- ☐ Kısmi ve tam iade kaydoluyor.
- ☐ İade tutarı snapshot fiyatından hesaplanıyor.
- ☐ Over-return engelleniyor.

Bu adımda öğrenilecekler

Service • Transaction • Conflict • Concurrency

ADIM 22 - İade Senaryolarını Doğrula

FAZ

Faz 5 - İade Sistemi

ÖN KOŞUL

Adım 21

Bu adımda ne yapıyoruz?

İade sisteminin sınır durumlarda yanlış miktar veya tutar üretmediğini test ediyoruz.

Yapılacaklar

- [] Beş adet satılan üründen iki adet iade et.
- [] Kalan üç ürünü ikinci işlemde iade et.
- [] Tamamı iade edilmiş satıra yeni iade göndermeyi dene.
- [] İki iade sonrası satın alınandan fazla olacak miktar gönder.
- [] Quantity değerini 0 ve negatif gönder.
- [] Olmayan OrderItem ile iade oluşturmayı dene.
- [] Product fiyatını değiştir ve iade tutarının eski OrderItem fiyatından hesaplandığını doğrula.
- [] İade nedeninin database'de kaydedildiğini kontrol et.
- [] İade toplamlarının OrderItem.Quantity değerini aşmadığını SQL veya test ile doğrula.
- [] Aynı anda iki iade isteği için belirlenen korumanın çalıştığını integration testte kontrol et.

Dikkat edilmesi gerekenler

- Yalnız tek iade senaryosuyla yetinme.
- Test verisini production database üzerinde oluşturma.

Bu adım bittiğinde

- ☐ Partial, full, repeated ve over-return senaryoları güvenlidir.
- ☐ İade tutarı finansal snapshot ile uyumludur.

Bu adımda öğrenilecekler

Boundary testing • Integration test • Data integrity

ADIM 23 - Analytics Tarih ve Response Temelini Oluştur

FAZ

Faz 6 - Analytics

ÖN KOŞUL

Sipariş ve iade verisi hazır

Bu fazın hedefi

AI eklemenden önce gerçek veriden doğru ve test edilebilir BI hesapları üret.

Bu adımda ne yapıyoruz?

Bütün analizlerin aynı tarih aralığı, filtre ve grafik response kurallarını kullanmasını sağlıyoruz.

Yapılacaklar

- [] AnalyticsService interface ve implementation oluştur.
- [] Tarih aralığını başlangıç dahil, bitiş hariç olacak şekilde standartlaştır.
- [] Bütün analytics tarihlerinde UTC kullan.
- [] Başlangıç tarihinin bitişten önce olmasını doğrula.
- [] Aşırı geniş tarih aralığı için makul backend sınırı belirle.
- [] Günlük, haftalık ve aylık seçenekleri için anlaşılır enum oluştur.
- [] Grafikler için periodStart, label ve numeric value içeren ortak model oluştur.
- [] Para, adet ve yüzde alanlarını JSON içinde numeric döndür.
- [] Boş tarih aralığında hata yerine tanımlı boş/sıfır response politikası uygula.
- [] Analytics servisinin AI servisine bağlı olmadığını doğrula.

Dikkat edilmesi gerekenler

- DateTime.Now kullanımını farklı dosyalara dağıtma.
- Ay isimlerine göre database grouping yapma.

Bu adım bittiğinde

- ☐ Bütün analizler ortak tarih ve chart contractını kullanabilir.
- ☐ AI olmadan analytics servisi çalışır.

Bu adımda öğrenilecekler

Date range • UTC • DTO • Chart data

ADIM 24 - Sales Summary Analizini Oluştur

FAZ

Faz 6 - Analytics

ÖN KOŞUL

Adım 23

Bu adımda ne yapıyoruz?

Seçilen dönem için temel satış KPI'larını gerçek siparişlerden hesaplıyoruz.

Yapılacaklar

- [] Analize hangi OrderStatus değerlerinin dahil olacağını açıkça belirle.
- [] Seçilen tarih aralığındaki toplam satış database'de SUM ile hesapla.
- [] Toplam sipariş sayısını hesapla.
- [] Satılan toplam ürün adedini OrderItem.Quantity üzerinden hesapla.
- [] Ortalama sipariş tutarını toplam satış / sipariş sayısı olarak hesapla.
- [] Sipariş sayısı sıfır olduğunda bölme hatasını engelle.
- [] Sonuç için SalesSummaryResponse oluştur.
- [] Sorgunun gereksiz tüm entityleri belleğe çekmediğini kontrol et.
- [] Bilinen küçük test verisiyle sonucu elle hesaplayıp karşılaştır.
- [] Boş dönem ve iptal edilmiş sipariş senaryosunu test et.

Dikkat edilmesi gerekenler

- Revenue veya average değerini kod içinde sabitleme.
- Canceled siparişleri yanlışlıkla satışa dahil etme.

Bu adım bittiğinde

- ☐ Revenue, order count, units sold ve average order value doğru hesaplanıyor.
- ☐ Boş dönem 500 hatası üretmiyor.

Bu adımda öğrenilecekler

SUM • COUNT • Average • LINQ aggregate

ADIM 25 - Sales Trend Analizini Oluştur

FAZ

Faz 6 - Analytics

ÖN KOŞUL

Adım 23-24

Bu adımda ne yapıyoruz?

Satışları günlük, haftalık veya aylık zaman serisine dönüştürüyoruz.

Yapılacaklar

- [] Kullanıcının seçtiği granularity değerini doğrula.
- [] Siparişleri günlük başlangıç tarihlerine göre grupta.
- [] Haftalık başlangıç gününü net olarak belirle ve grupta.
- [] Aylık gruplamada yıl ve ayı birlikte kullan.
- [] Her dönem için revenue, order count ve units sold hesapla.
- [] Sonuçları en eski dönemden en yeniye sırala.
- [] Satış olmayan ara dönemleri grafik için sıfır noktayla doldur.
- [] Her noktada machine-readable periodStart ve okunabilir label döndür.
- [] Ay sonu, yıl geçişi ve artık gün senaryolarını test et.
- [] EF Core sorgusunun PostgreSQL'e çevrildiğini doğrula; tüm siparişleri belleğe çekme.

Dikkat edilmesi gerekenler

- Dönem toplamlarını hazır liste olarak kodlama.
- Sadece ay adına göre gruplayıp farklı yılları birleştirme.

Bu adım bittiğinde

- ☐ Günlük, haftalık ve aylık trend grafiği datası oluşuyor.
- ☐ Noktalar doğru sırada ve eksik dönemler kontrollü.

Bu adımda öğrenilecekler

GROUP BY • Time series • Date bucket • Gap filling

ADIM 26 - Top Products Analizini Oluştur

FAZ

Faz 6 - Analytics

ÖN KOŞUL

Adım 23-24

Bu adımda ne yapıyoruz?

En çok satan ürünleri adet veya gelir ölçütüne göre dinamik olarak sıralıyoruz.

Yapılacaklar

- [] Tarih aralığındaki tamamlanmış OrderItem kayıtlarını kullan.
- [] Ürünleri ProductId bazında grupta.
- [] Satılan toplam Quantity değerini hesapla.
- [] Toplam geliri LineTotal üzerinden hesapla.
- [] Kullanıcının adet veya gelir kriteri seçebilmesini sağla.
- [] Sonuç sayısı için varsayılan ve maksimum limit uygula.
- [] İsteğe bağlı kategori filtresi ekle.
- [] Eşit sonuçlarda kararlı sıralama için ikinci bir kriter kullan.
- [] Response içinde rank, product id, name, SKU, quantity ve revenue döndür.
- [] Adet ve gelir sıralamalarının farklı olabildiği test verisi oluştur.
- [] Yeni büyük sipariş sonrası sıralamanın değişebildiğini doğrula.

Dikkat edilmesi gerekenler

- Laptop veya Telefon gibi hazır lider ürün yazma.
- Al'ın ürün sıralamasını belirlemesine izin verme.

Bu adım bittiğinde

- ☐ Top products adet ve gelir kriterine göre çalışıyor.
- ☐ Kategori ve limit filtreleri güvenli.

Bu adımda öğrenilecekler

Grouping • Ranking • Top-N • Filtering

ADIM 27 - Category Performance Analizini Oluştur

FAZ

Faz 6 - Analytics

ÖN KOŞUL

Adım 23-26

Bu adımda ne yapıyoruz?

Kategorilerin ne kadar gelir ve satış adedi ürettiğini karşılaştırıyoruz.

Yapılacaklar

- [] OrderItem kayıtlarını Product üzerinden Category ile birleştir.
- [] Her kategori için toplam revenue hesapla.
- [] Her kategori için satılan toplam ürün adedini hesapla.
- [] Genel revenue toplamını hesapla.
- [] Her kategorinin toplam satış içindeki yüzde payını backend'de hesapla.
- [] Toplam satış sıfırda yüzde hesaplamasında bölme hatasını engelle.
- [] Kategorileri revenue değerine göre sırala.
- [] Tarih aralığı filtresini uygula.
- [] Inactive olmuş ürün ve kategorilerin tarihsel satışlarını kaybetme.
- [] Bilinen test verisiyle kategori toplamlarının genel toplama eşit olduğunu doğrula.

Dikkat edilmesi gerekenler

- Kategori paylarını hazır yüzde olarak seed veya kod içine yazma.
- Inactive kayıtları tarihsel analytics'ten otomatik silme.

Bu adım bittiğinde

- ☐ Kategori revenue, units ve toplam içindeki payı doğru hesaplanıyor.
- ☐ Kategori toplamları genel toplamla tutarlı.

Bu adımda öğrenilecekler

Join • Category aggregate • Percentage share

ADIM 28 - Önceki Dönem Karşılaştırmasını Ekle

FAZ

Faz 6 - Analytics

ÖN KOŞUL

Sales Summary ve Category Performance hazır

Bu adımda ne yapıyoruz?

Bu dönem ile aynı uzunluktaki önceki dönemi karşılaştırıp değişim yüzdesini backend'de hesaplıyoruz.

Yapılacaklar

- [] Kullanıcının seçtiği dönem uzunluğunu hesapla.
- [] Bitişik ve aynı uzunlukta önceki tarih aralığını oluştur.
- [] Mevcut dönem satışını hesapla.
- [] Önceki dönem satışını aynı kurallarla hesapla.
- [] Değişim yüzdesini (mevcut - önceki) / önceki x 100 formülüyle hesapla.
- [] Önceki değer sıfır olduğunda sonsuz veya hatalı yüzde üretme.
- [] Artış, azalış, değişim yok ve karşılaştırma tabanı yok durumlarını açıkça işaretle.
- [] Aynı karşılaştırmayı kategori performansında yeniden kullan.
- [] 500.000 ve 600.000 örneğini yaklaşık -%16,67 olarak test et.
- [] Ay uzunlukları farklı olduğunda kullanılan dönem politikasını response veya dokümanda açıkla.

Dikkat edilmesi gerekenler

- Yüzde değişimini Al'a hesaplatma.
- Önceki değer sıfırken rastgele yüzde döndürme.

Bu adım bittiğinde

- ☐ Backend dönem karşılaştırmasını güvenli hesaplıyor.
- ☐ Zero baseline durumu açıklanabilir.

Bu adımda öğrenilecekler

Period comparison • Percentage change • Zero denominator

ADIM 29 - Return Analytics'i Oluştur

FAZ

Faz 6 - Analytics

ÖN KOŞUL

Return sistemi ve Adım 23 hazır

Bu adımda ne yapıyoruz?

İade hacmini, tutarını, oranını, nedenlerini ve en çok iade edilen ürünleri hesaplıyoruz.

Yapılacaklar

- ☐ Seçilen dönemdeki iade kayıtlarının sayısını hesapla.
- ☐ İade edilen toplam ürün adedini hesapla.
- ☐ Toplam iade tutarını Return.Amount üzerinden hesapla.
- ☐ İade oranının pay ve payda tanımını açıkça belirle.
- ☐ Satış miktarı sıfırda oran hesaplamasını güvenli yönet.
- ☐ İadeleri Reason alanına göre grupla.
- ☐ En fazla iade edilen ürünleri Quantity bazında sırala.
- ☐ İade tarihi ile sipariş tarihi arasındaki filtre farkını dokümente et.
- ☐ Kısmi ve birden fazla iade kayıtlarını doğru topla.
- ☐ Response içinde kullanılan iade oranı tanımını açıklayan kısa metadata döndür.
- ☐ Bilinen satış/iade verisiyle oranı elle doğrula.

Dikkat edilmesi gerekenler

- İade oranı ve nedenlerini kod içinde sabitleme.
- Product'ın güncel fiyatını iade tutarında kullanma.

Bu adım bittiğinde

- ☐ Return count, quantity, amount, rate, reasons ve products doğru hesaplanıyor.
- ☐ Boş denominator güvenli.

Bu adımda öğrenilecekler

Ratio • Grouping • Return metrics • Business definition

ADIM 30 - Analytics Endpointlerini Yayınla ve Sorguları Kontrol Et

FAZ

Faz 6 - Analytics

ÖN KOŞUL

Adım 24-29

Bu adımda ne yapıyoruz?

Bütün analizleri AI'dan bağımsız REST endpointleri olarak sunup performanslarını kontrol ediyoruz.

Yapılacaklar

- [] GET /api/analytics/summary endpointini oluştur.
- [] GET /api/analytics/sales-trend endpointini oluştur.
- [] GET /api/analytics/top-products endpointini oluştur.
- [] GET /api/analytics/categories endpointini oluştur.
- [] GET /api/analytics/returns endpointini oluştur.
- [] Tarih, granularity, metric ve limit parametrelerini validate et.
- [] Bütün endpointlerde aynı tarih ve error contractını kullan.
- [] Response'ların grafik çizmeye uygun JSON döndürdüğünü kontrol et.
- [] EF Core'un ürettiği SQL'i incele.
- [] N+1 veya tüm tabloyu belleğe alma problemi olmadığını kontrol et.
- [] OrderDate, ReturnDate ve foreign key indexlerini gerçek sorgu planına göre gözden geçir.
- [] Seed database üzerinde beş endpointi Swagger'dan test et.

Dikkat edilmesi gerekenler

- Analytics endpointlerini AI servisine bağımlı yapma.
- Sınırsız tarih aralığı veya limit kabul etme.

Bu adım bittiğinde

- ☐ Beş analytics endpointi AI key olmadan çalışıyor.
- ☐ Sorgular dinamik, sınırlı ve chart-ready.

Bu adımda öğrenilecekler

REST analytics • Query plan • Index • Performance

ADIM 31 - Provider Bağımsız AI Servisini Kur

FAZ

Faz 7 - AI Entegrasyonu

ÖN KOŞUL

Analytics endpointleri hazır

Bu fazın hedefi

AI yalnız soruyu anlasın ve sonucu açıklasın; database gerçeğinin yerine geçmesin.

Bu adımda ne yapıyoruz?

Gemini veya OpenAI değişebilse de projenin geri kalanını provider'a bağlamayan sade bir AI katmanı kuruyoruz.

Yapılacaklar

- [] IAIService interface oluştur.
- [] Kullanıcı sorusunu anlamak ve sonucu açıklamak için iki ayrı metod tanımla.
- [] Seçilecek provider için bir adapter implementation oluştur.
- [] Provider, model, API key ve timeout değerlerini configurationdan oku.
- [] AI API key'i user-secrets veya environment variable üzerinden al.
- [] HttpClient veya provider SDK'sını Dependency Injection ile kaydet.
- [] Timeout, cancellation ve geçici provider hatalarını ele al.
- [] Provider response tiplerini feature dışına sızdırma.
- [] Mock/fake AI implementation ile gerçek API çağrısı olmadan test edilebilirlik sağla.
- [] AI isteğinin süresini logla; key ve tam hassas promptu loglama.

Dikkat edilmesi gerekenler

- API key'i source code'a yazma.
- Business servislerini doğrudan Gemini/OpenAI SDK tiplerine bağlama.

Bu adım bittiğinde

- ☐ Provider adapter değiştirilebilir.
- ☐ AI servisi mock ile test edilebilir.
- ☐ Secret güvenli configurationdan gelir.

Bu adımda öğrenilecekler

Interface • Adapter • HttpClient • External API

ADIM 32 - Kullanıcının Sorusunu AI ile Anlamlandır

FAZ

Faz 7 - AI Entegrasyonu

ÖN KOŞUL

Adım 31

Bu adımda ne yapıyoruz?

Kullanıcı sorusundan hangi analizin, tarihin, filtrenin ve sonucun istendiğini yapılandırılmış biçimde çıkarıyoruz.

Yapılacaklar

- [] AI'a QueryPilot'ın desteklediği analizleri açıkça bildir.
- [] Satış özeti, satış trendi, top products, kategori performansı ve iade analizi seçeneklerini tanımla.
- [] Kullanıcının hangi analizi istediğini belirlemesini iste.
- [] Soruda tarih veya dönem varsa ayrı alan olarak çıkarmasını sağla.
- [] Soruda ürün veya kategori adı varsa ayrı parametre olarak çıkarmasını sağla.
- [] Top products sorusunda adet veya gelir kriterini çıkarmasını sağla.
- [] Kullanıcı sonuç sayısı söylediye limit değerini çıkarmasını sağla.
- [] AI'dan yalnız yapılandırılmış JSON cevap iste.
- [] Raw SQL veya serbest business cevabı isteme.
- [] 'Son 3 ayda en çok satan 5 ürün ne?' sorusuyla analiz, dönem ve limit çıktısını doğrula.
- [] Farklı Türkçe soru kalıplarıyla deneme yap.

Dikkat edilmesi gerekenler

- question.Contains ile hazır cevap eşleştirmesi yapma.
- AI'dan satış rakamı veya ürün sıralaması isteme.

Bu adım bittiğinde

- ☐ Doğal dil sorusu desteklenen analiz ve parametrelere dönüşüyor.
- ☐ AI henüz database sonucunu üretmiyor.

Bu adımda öğrenilecekler

Structured output • Intent • Parameter extraction • Prompt design

ADIM 33 - AI Çıktısını Backend İçinde Doğrula

FAZ

Faz 7 - AI Entegrasyonu

ÖN KOŞUL

Adım 32

Bu adımda ne yapıyoruz?

AI cevabını güvenilir kabul etmeyip analytics çalışmadan önce backend kurallarıyla kontrol ediyoruz.

Yapılacaklar

- [] AI JSON cevabını typed bir modele dönüştür.
- [] Tanımlanmayan analiz türünü reddet.
- [] Her analiz için gerekli alanların bulunup bulunmadığını kontrol et.
- [] Tarih aralığının geçerli ve backend sınırları içinde olduğunu doğrula.
- [] Granularity değerini izin verilen seçeneklerle sınırla.
- [] Metric değerini adet veya gelir seçenekleriyle sınırla.
- [] Limit değerini minimum ve maksimum backend sınırına uygula.
- [] Kategori parametresi varsa database'de gerçekten bulunup bulunmadığını kontrol et.
- [] AI cevabında raw SQL alanı veya beklenmeyen komut varsa çalıştırma.
- [] Geçersiz AI çıktısı durumunda analytics servisini çağırma.
- [] Malformed JSON, unknown analysis, invalid date ve çok yüksek limit senaryolarını test et.

Dikkat edilmesi gerekenler

- AI çıktısını validation olmadan service metoduna gönderme.
- AI'ın belirlediği sınırsız limit veya tarihi kabul etme.

Bu adım bittiğinde

- ☐ Yalnız tanımlı ve geçerli AI sonuçları analytics'e ulaşır.
- ☐ Geçersiz çıktı güvenli cevap üretir.

Bu adımda öğrenilecekler

Trust boundary • Schema validation • Allow-list • Fail closed

ADIM 34 - Eksik veya Belirsiz Soruları Yönet

FAZ

Faz 7 - AI Entegrasyonu

ÖN KOŞUL

Adım 32-33

Bu adımda ne yapıyoruz?

Sistem, cevap için gerekli bilgi eksikse yanlış tahmin yapmak yerine kullanıcıdan netleştirme ister.

Yapılacaklar

- [] Sorunun analiz için gerekli bilgileri içerip içermediğini kontrol et.
- [] Eksik bilgi varsa database analytics sorgusu çalıştırma.
- [] Kullanıcıya hangi bilginin eksik olduğunu açıkça söyle.
- [] 'En iyi ürünler hangileri?' sorusunda satış adedi mi gelir mi istediğini sor.
- [] Tarih zorunluysa kullanıcıdan tarih aralığını isteme davranışını ekle.
- [] Belirsiz kategori veya ürün adı varsa rastgele eşleşme seçme.
- [] Kullanıcıdan beklenen bilgiyi response içinde anlaşılır metin ve alanlarla göster.
- [] Yeni bilgiyle sorunun tekrar gönderilebilmesini kolaylaştıracak response yapısı hazırla.
- [] Belirsiz sorularda rastgele tarih, kategori veya metric seçilmediğini test et.
- [] Bu durumda PostgreSQL'e analytics sorgusu gitmediğini test/mock ile doğrula.

Dikkat edilmesi gerekenler

- Eksik parametreyi business varsayımıyla doldurma.
- Belirsiz soruya sahte başarılı cevap verme.

Bu adım bittiğinde

- ☐ Sistem bilmediğini uydurmuyor.
- ☐ Kullanıcıya kısa ve açık netleştirme sorusu dönüyor.

Bu adımda öğrenilecekler

Ambiguity • Clarification • Safe defaults • User experience

ADIM 35 - Gerçek Analytics Sonucunu AI ile Açıkla

FAZ

Faz 7 - AI Entegrasyonu

ÖN KOŞUL

Adım 31 ve Analytics hazır

Bu adımda ne yapıyoruz?

Backend'in hesapladığı gerçek sayıları değiştirmeden kullanıcıya sade bir açıklama üretiriyoruz.

Yapılacaklar

- [] Analytics DTO'sunu explanation isteğinde AI'a gönder.
- [] AI'a yalnız verilen sayıları kullanmasını açıkça söyle.
- [] Yeni ürün, müşteri, tarih veya yüzde eklememesini iste.
- [] Artış, azalış ve öne çıkan noktayı kısa Türkçe ile açıklamasını sağla.
- [] Raw analytics verisini AI açıklamasından bağımsız olarak response'ta koru.
- [] AI açıklaması başarısız olursa numeric sonucu kaybetme.
- [] Açıklama uzunluğu için makul sınır belirle.
- [] 610.000 ve 540.000 gibi test fixture'ı ile azalış açıklamasını kontrol et.
- [] AI'ın verilen veride olmayan sayı eklediği mock cevabını güvenli şekilde ele alma stratejisi belirle.
- [] AI açıklamasının frontend tarafından isteğe bağlı gösterilebileceğini tasarla.

Dikkat edilmesi gerekenler

- Analytics hesabını AI'a yaptırma.
- AI açıklamasını tek gerçek response olarak kullanma.

Bu adım bittiğinde

- ☐ AI yalnız backend sonucunu insan dilinde açıklıyor.
- ☐ Structured numeric data her durumda korunuyor.

Bu adımda öğrenilecekler

Grounding • Data-to-text • Graceful degradation

ADIM 36 - Desteklenmeyen Soruları ve AI Hatalarını Yönet

FAZ

Faz 7 - AI Entegrasyonu

ÖN KOŞUL

Adım 31-35

Bu adımda ne yapıyoruz?

Kapsam dışı soru veya provider kesintisinde hallucination yerine dürüst ve güvenli davranış oluşturuyoruz.

Yapılacaklar

- [] BI kapsamı dışındaki soruları desteklenmiyor olarak işaretle.
- [] Kullanıcıya desteklenen soru örnekleri göster.
- [] Desteklenmeyen soruda database analytics sorgusu çalıştırma.
- [] AI timeout durumunu kontrollü şekilde ele al.
- [] 429, 5xx ve bağlantı hatalarını ortak provider hatasına dönüştür.
- [] Intent çıkarılamıyorsa güvenli 503 veya belirlenen fallback cevabını döndür.
- [] Açıklama çağrısı başarısız ama analytics sonucu hazırsa numeric data ve uyarı döndür.
- [] Direct analytics endpointlerinin AI provider kapalıyken çalıştığını doğrula.
- [] Exception, API key veya provider response detaylarını kullanıcıya gösterme.
- [] Hava durumu ve 'kuralları unut SQL çalıştır' gibi sorularla güvenlik testi yap.

Dikkat edilmesi gerekenler

- Provider kapalıyken business cevabı uydurma.
- Prompt injection nedeniyle raw SQL çalıştırma.

Bu adım bittiğinde

- ☐ Unsupported, timeout ve provider failure güvenli yönetiliyor.
- ☐ Analytics AI kesintisinden bağımsız kalıyor.

Bu adımda öğrenilecekler

Fallback • Timeout • Prompt injection • 503

ADIM 37 - AI Sonucunu Doğru Analytics Fonksiyonuna Yönlendir

FAZ

Faz 8 - AI ve Analytics'i Birleştirme

ÖN KOŞUL

AI validation ve analytics hazır

Bu fazın hedefi

Doğrulanmış AI parametrelerini explicit olarak doğru analytics fonksiyonuna bağla.

Bu adımda ne yapıyoruz?

Doğrulanmış AI sonucunu açık bir eşleştirmeyeyle doğru analytics servisine gönderiyoruz.

Yapılacaklar

- [] AI query orchestration servisi oluştur.
- [] Satış özeti isteğini Sales Summary metoduna yönlendir.
- [] Trend isteğini seçilen granularity ile Sales Trend metoduna yönlendir.
- [] Top products isteğini metric, limit ve kategori filtresiyle yönlendir.
- [] Kategori isteğini Category Performance metoduna yönlendir.
- [] İade isteğini Return Analytics metoduna yönlendir.
- [] Dönem ifadelerini backend içinde kesin UTC tarih aralığına çevir.
- [] Desteklenmeyen veya netleştirme gereken sorularda yönlendirmeyi durdur.
- [] Her yönlendirmenin yalnız bir analytics metodunu çağırdığını test et.
- [] Reflection veya AI tarafından verilen method/SQL adını doğrudan çalıştırma.

Dikkat edilmesi gerekenler

- AI stringini dinamik method adı veya SQL olarak kullanma.
- Validationdan geçmeyen isteği route etme.

Bu adım bittiğinde

- ☐ Her desteklenen soru doğru analytics fonksiyonuna gider.
- ☐ Yönlendirme açık ve test edilebilirdir.

Bu adımda öğrenilecekler

Orchestration • Routing • Date resolution • Allow-list

ADIM 38 - AI Query Endpointini Oluştur

FAZ Faz 8 - AI ve Analytics'i Birleştirme ÖN KOŞUL Adım 37

Bu adımda ne yapıyoruz?

Frontend'in tek bir doğal dil sorusu göndererek intent, gerçek data ve açıklama alacağı endpointi oluşturuyoruz.

Yapılacaklar

- [] POST /api/ai/query request modeli oluştur.
- [] Question alanının boş olmamasını ve maksimum uzunluğunu validate et.
- [] Endpoint üzerinden AI intent extraction çağrısını yap.
- [] Backend validation ve clarification adımlarını uygula.
- [] Doğru analytics servisini çağır.
- [] Gerçek analytics sonucunu AI explanation servisine gönder.
- [] Response içinde analiz türü, kullanılan parametreler, structured data, chart data ve açıklama döndür.
- [] Netleştirme gereken durumda uygun durum ve soru döndür.
- [] Provider tamamen kullanılamıyorsa 503/fallback politikasını uygula.
- [] Endpointi Swagger'da anlaşılır request/response açıklamasıyla göster.
- [] Boş soru, geçerli soru, belirsiz soru ve unsupported soru testlerini yap.

Dikkat edilmesi gerekenler

- Endpoint içine analytics hesabı yazma.
- Yalnız AI metnini döndürüp raw datayı kaybetme.

Bu adım bittiğinde

- ☐ Tek endpoint doğal dil sorusunu güvenli BI cevabına dönüştürüyor.
- ☐ Frontend structured data ve açıklamayı birlikte alıyor.

Bu adımda öğrenilecekler

API orchestration • Response design • Validation

ADIM 39 - AI ve Analytics Akışını Uçtan Uca Doğrula

FAZ

Faz 8 - AI ve Analytics'i Birleştirme

ÖN KOŞUL

Adım 38

Bu adımda ne yapıyoruz?

Kullanıcı sorusundan frontend response'una kadar bütün zincirin gerçek database verisiyle çalıştığını kanıtlıyoruz.

Yapılacaklar

- [] 'Bu ay satışlarımız nasıl?' sorusunu gönder.
- [] 'Son 3 ayda en çok satan 5 ürün ne?' sorusunu gönder.
- [] Kategori ve önceki dönem karşılaştırması isteyen bir soru gönder.
- [] İade analizi isteyen bir soru gönder.
- [] Her soruda AI'ın çıkardığı parametreleri kontrol et.
- [] Çalışan analytics metodunu log/test ile doğrula.
- [] Response numeric değerlerini doğrudan analytics endpointiyle karşılaştır.
- [] AI açıklamasının bu değerlerle çelişmediğini kontrol et.
- [] Belirsiz soruda netleştirme cevabı ve sıfır analytics çağırısı doğrula.
- [] Provider kapalıyken direct analytics'in çalıştığını tekrar kontrol et.
- [] Akışı dokümana basit bir diyagram olarak ekle.

Dikkat edilmesi gerekenler

- AI cevabını database sonucu kontrol edilmeden kabul etme.
- Sadece tek mutlu soru ile test yapma.

Bu adım bittiğinde

- ☐ Soru -> AI -> validation -> analytics -> DB -> açıklama akışı çalışıyor.
- ☐ Bütün sayılar PostgreSQL sonucuyla eşleşiyor.

Bu adımda öğrenilecekler

End-to-end flow • Grounding • Traceability

ADIM 40 - Sipariş ve İade Unit Testlerini Yaz

FAZ

Faz 9 - Testler

ÖN KOŞUL

Order ve Return servisleri hazır

Bu fazın hedefi

Kritik kuralları otomatik testlerle tekrar üretilebilir biçimde kanıtla.

Bu adımda ne yapıyoruz?

En kritik finansal kuralları hızlı ve tekrarlanabilir testlerle koruyoruz.

Yapılacaklar

- [] Unit test projesini solution'a ekle.
- [] OrderItem LineTotal hesaplamasını test et.
- [] Order.TotalAmount hesaplamasını test et.
- [] Frontend fiyat/toplam gönderse bile kullanılmadığını test et.
- [] Product fiyatı değişince eski snapshotın değişmediğini test et.
- [] Olmayan ve pasif müşteri/ürün durumlarını test et.
- [] Quantity 0 ve negatif durumlarını test et.
- [] Partial ve full return testlerini yaz.
- [] Daha önceki iadeler dahil over-return testini yaz.
- [] Return Amount değerinin OrderItem fiyatından hesaplandığını test et.
- [] Invalid durumda SaveChanges çağrılmadığını doğrula.
- [] Test isimlerini senaryoyu okuyarak anlayacak şekilde yaz.

Dikkat edilmesi gerekenler

- Sadece mock interaction test edip gerçek hesabı atlama.
- Production kodunu test geçsin diye gevşetme.

Bu adım bittiğinde

- ☐ Sipariş ve iade invariantları otomatik testlerle korunuyor.
- ☐ Kural bozulursa ilgili test fail oluyor.

Bu adımda öğrenilecekler

Unit test • AAA • Test fixture • Invariant

ADIM 41 - Analytics ve PostgreSQL Integration Testlerini Yaz

FAZ

Faz 9 - Testler

ÖN KOŞUL

Analytics endpointleri hazır

Bu adımda ne yapıyoruz?

EF Core sorgularının gerçek PostgreSQL üzerinde doğru SQL ve doğru sonuç ürettiğini kanıtıyoruz.

Yapılacaklar

- [] Ayrı integration test projesi veya bölümü oluştur.
- [] Disposable test PostgreSQL database/container hazırla.
- [] Migrationları test database'ine uygula.
- [] Küçük ve elle hesaplanabilir order/return fixture'ı oluştur.
- [] Sales Summary değerlerini elle hesaplanan sonuçla karşılaştır.
- [] Günlük, haftalık ve aylık trend bucketlarını test et.
- [] Top Products adet ve gelir sıralamasını test et.
- [] Category share ve previous period yüzdesini test et.
- [] Return rate, reasons ve most returned products sonuçlarını test et.
- [] Boş dönem, yıl geçişi ve zero denominator durumlarını test et.
- [] Testlerin development veya production database'ine bağlanmasını engelle.
- [] Test sonrası veriyi güvenli şekilde temizle veya test containerını kaldır.

Dikkat edilmesi gerekenler

- EF InMemory sonucunu PostgreSQL doğruluğu olarak kabul etme.
- Paylaşılan gerçek database üzerinde destructive test çalıştırma.

Bu adım bittiğinde

- ☐ Analytics formülleri gerçek PostgreSQL üzerinde doğrulanıyor.
- ☐ Migration ve LINQ translation sorunları testte yakalanıyor.

Bu adımda öğrenilecekler

Integration test • Test database • SQL translation • Fixture

ADIM 42 - AI Contract ve API Testlerini Yaz

FAZ Faz 9 - Testler ÖN KOŞUL AI query endpointi hazır

Bu adımda ne yapıyoruz?

Canlı AI maliyetine bağımlı olmadan AI sözleşmesini ve HTTP davranışını test ediyoruz.

Yapılacaklar

- [] Fake AI provider oluştur.
- [] Geçerli sales summary, trend, top products, category ve return intent cevaplarını test et.
- [] Malformed JSON cevabını test et.
- [] Tanımlanmayan analiz türünü test et.
- [] Eksik tarih, metric ve limit senaryolarını test et.
- [] Belirsiz soruda analytics çağrılmadığını doğrula.
- [] Provider timeout ve hata fallbackini test et.
- [] AI'nın raw SQL benzeri alan döndürdüğü cevabın çalıştırılmadığını test et.
- [] WebApplicationFactory ile Category, Product, Order, Return ve Analytics endpointlerini HTTP üzerinden test et.
- [] 400, 404, 409, 500 ve 503 response formatlarını kontrol et.
- [] Gerçek AI çağrısını yalnız ayrı ve isteğe bağlı smoke test olarak tut.

Dikkat edilmesi gerekenler

- Unit/integration testlerde her zaman gerçek ücretli AI servisini çağırma.
- Invalid AI cevabının analytics'e ulaşmasına izin verme.

Bu adım bittiğinde

- ☐ AI trust boundary ve API contract otomatik testlidir.
- ☐ Testler provider internet erişimi olmadan çalışabilir.

Bu adımda öğrenilecekler

Contract test • Fake provider • API integration • WebApplicationFactory

ADIM 43 - Sistemin Hardcoded Olmadığını Dinamik Veriyle Kanıtla

FAZ

Faz 10 - Dinamik Çalışma Kanıtı

ÖN KOŞUL

Analytics ve AI akışı testli

Bu fazın hedefi

Database değiştiğinde sonuçların değiştiğini göstererek hardcoded olmadığını kanıtla.

Bu adımda ne yapıyoruz?

Database'e yeni sipariş ekleyip aynı soruların yeni sonuç üretmesini gösteriyoruz.

Yapılacaklar

- [] Top Products sonucunu başlangıç değeri olarak kaydet.
- [] Sales Summary ve Category Performance başlangıç değerlerini kaydet.
- [] Database'deki aktif bir ürünü seç.
- [] Bu ürünün satışını ciddi şekilde artıracak yeni siparişler oluştur.
- [] Aynı tarih aralığıyla Top Products sorgusunu tekrar çalıştır.
- [] Ürün sıralamasının yeni database kayıtlarına göre değiştiğini doğrula.
- [] Sales Summary revenue artışının yeni order toplamalarına eşit olduğunu doğrula.
- [] Kategori revenue ve payının değiştiğini doğrula.
- [] Aynı doğal dil sorusunu AI endpointine tekrar gönder.
- [] AI'nın eski sonucu değil, yeni analytics sonucunu açıkladığını doğrula.
- [] Canceled veya invalid orderın analytics'i değiştirmedeğini test et.
- [] Bu akışı CV demosunda kullanılacak kısa bir senaryo olarak kaydet.

Dikkat edilmesi gerekenler

- Testte belirli ürünün mutlaka lider olacağını production koduna yazma.
- Beklenen yeni sonucu sabit revenue değeriyle karşılaştırma; eklenen order delta'sından hesapla.

Bu adım bittiğinde

- ☐ Revenue, ranking ve category sonuçları database değişimiyle güncelleniyor.
- ☐ AI yeni sonucu açıklıyor.
- ☐ Hardcode olmadığı görünür şekilde kanıtlanıyor.

Bu adımda öğrenilecekler

Mutation test • Dynamic data • Demo proof • Grounding

ADIM 44 - Swagger ve CORS Ayarlarını Tamamla

FAZ

Faz 11 - Frontend İçin Hazırlık

ÖN KOŞUL

Bütün endpointler hazır

Bu fazın hedefi

Frontend seçmeden temiz, belgeli ve teknoloji bağımsız REST sözleşmesi hazırla.

Bu adımda ne yapıyoruz?

Frontend seçilmeden API'nin keşfedilebilir ve güvenli şekilde çağrılabilir olmasını sağlıyoruz.

Yapılacaklar

- [] Swagger başlığını ve proje açıklamasını düzenle.
- [] Endpointleri Categories, Products, Customers, Orders, Returns, Analytics ve AI etiketleriyle grupla.
- [] Request, response ve hata tiplerinin Swagger'da doğru görünmesini sağla.
- [] Enum seçeneklerini anlaşılır göster.
- [] Örneklerin sadece dokümantasyon örneği olduğunu belirt.
- [] CORS originlerini configuration üzerinden oku.
- [] Development frontend originini ekle.
- [] Production'da exact HTTPS origin listesi kullan.
- [] Credentials kullanılıyorsa wildcard origin açma.
- [] Allowed ve disallowed origin preflight testlerini yap.
- [] Production'da Swagger'ın açık/kapalı olma politikasını belirle.

Dikkat edilmesi gerekenler

- Production CORS'u AllowAnyOrigin ile kontrolsüz açma.
- Swagger örneğine gerçek API key veya customer verisi koyma.

Bu adım bittiğinde

- ☐ Frontend API'yi Swagger üzerinden anlayabiliyor.
- ☐ Yalnız izin verilen originler CORS cevabı alıyor.

Bu adımda öğrenilecekler

OpenAPI • CORS • Preflight • API discoverability

ADIM 45 - Frontend İçin JSON ve Error Sözleşmesini Temizle

FAZ Faz 11 - Frontend İçin Hazırlık ÖN KOŞUL Adım 44

Bu adımda ne yapıyoruz?

React, Vue veya Angular'ın aynı düzenli REST response'larını kullanabilmesi için son contract kontrolünü yapıyoruz.

Yapılacaklar

- [] JSON alan adlarında camelCase kullan.
- [] Tarihleri ISO 8601 UTC biçiminde döndür.
- [] Para ve chart value alanlarını numeric tut.
- [] Liste endpointlerinde items, page, pageSize, totalCount ve totalPages alanlarını standartlaştır.
- [] Bütün error response'larının ProblemDetails biçiminde olduğunu kontrol et.
- [] Chart response'larında periodStart, label ve value alanlarını tutarlı kullan.
- [] AI response içinde structured data ile explanationı ayrı alanlarda döndür.
- [] Entitylerin hiçbir endpointte doğrudan serialize edilmediğini kontrol et.
- [] Pagination ve analytics limitlerini aşan requestleri test et.
- [] Basit bir HTTP client veya Postman collection ile API'nin frontend olmadan tamamen çalıştığını doğrula.

Dikkat edilmesi gerekenler

- Frontend'e özel component veya chart library kodunu backend'e ekleme.
- Formatted para stringini tek numeric kaynak olarak döndürme.

Bu adım bittiğinde

- ☐ API contractları tutarlı ve frontend bağımsızdır.
- ☐ Error, pagination ve chart verileri standarttır.

Bu adımda öğrenilecekler

JSON contract • Serialization • Frontend independence

ADIM 46 - README, Demo Verisi ve Demo Sorularını Tamamla

FAZ

Faz 11 - Frontend İçin Hazırlık

ÖN KOŞUL

Adım 43-45

Bu adımda ne yapıyoruz?

Başka bir kişinin projeyi sıfırdan kurup çalıştırabileceği ve kısa sürede gösterebileceği rehberi hazırlıyoruz.

Yapılacaklar

- [] README'e projenin kısa açıklamasını ve mimari diyagramını ekle.
- [] .NET, PostgreSQL ve diğer gereksinimleri yaz.
- [] User-secrets ile connection string ve AI key ayarlama komutlarını ekle.
- [] Migration, seed, run ve test komutlarını sırayla yaz.
- [] Ana endpointleri kısa bir tabloda göster.
- [] AI'ın sayı üretmediğini ve raw SQL çalıştırmadığını açıkla.
- [] Demo seed verisini bütün analytics sonuçlarını gösterecek şekilde son kez kontrol et.
- [] Sales summary, trend, top products, category, return, clarification ve unsupported örnek soruları yaz.
- [] Dinamik veri testi için 5-7 dakikalık demo akışı ekle.
- [] Known limitations ve Nice to Have listesini düzleştir.
- [] Fresh clone ile README adımlarını baştan sona dene.

Dikkat edilmesi gerekenler

- README'e gerçek secret ekleme.
- Çalışmayan komut veya uygulanmamış özelliği varmış gibi yazma.

Bu adım bittiğinde

- ☐ Yeni geliştirici projeyi README ile kurabiliyor.
- ☐ CV demosu tekrar üretilebilir.

Bu adımda öğrenilecekler

Dokümantasyon • Onboarding • Demo tasarımı • Reproducibility

ADIM 47 - Cloud PostgreSQL ve Production Backend'i Yayınla

FAZ

Faz 12 - Deployment

ÖN KOŞUL

Tüm Must Have geliştirme ve testler hazır

Bu fazın hedefi

Secrets, migration ve smoke test kurallarıyla çalışan sistemi production ortamına taşı.

Bu adımda ne yapıyoruz?

Yerel sistemi cloud PostgreSQL ve HTTPS backend ortamına güvenli configuration ile taşıyoruz.

Yapılacaklar

- [] Uygun bir managed PostgreSQL servisi oluştur.
- [] Database region, TLS, backup ve bağlantı limitlerini ayarla.
- [] Uygulama ve migration için minimum yetkili database kullanıcılarını hazırla.
- [] Production connection stringini host secret store veya environment variable olarak ekle.
- [] Backend için production host seç ve release build oluştur.
- [] ASPNETCORE_ENVIRONMENT değerini Production yap.
- [] Production CORS originlerini environment variable ile ayarla.
- [] AI provider, model ve API key ayarlarını environment variable/secret store ile ekle.
- [] Migrationı kontrollü deployment adımı olarak çalıştır.
- [] Demo seed çalışmasını production'da varsayılan kapalı tut; gerekiyorsa ayrı demo environment kullan.
- [] Backend'i HTTPS üzerinden yayınla.
- [] Deployment loglarında connection string veya API key olmadığını kontrol et.

Dikkat edilmesi gerekenler

- Secretları Docker image, repo veya CI loguna gömme.
- Production database'de otomatik ve kontrolsüz demo seed çalıştırma.

Bu adım bittiğinde

- ☐ Cloud database TLS ile erişiliyor.
- ☐ Production API secretsız config ile çalışıyor.
- ☐ Migration uygulanmıştır.

Bu adımda öğrenilecekler

Cloud PostgreSQL • Environment variables • TLS • Release build

ADIM 48 - Production Smoke Test, Güvenlik Kontrolü ve v1.0.0 Release

FAZ

Faz 12 - Deployment

ÖN KOŞUL

Adım 47

Bu adımda ne yapıyoruz?

Yayınlanan sistemin kritik akışlarını kontrol edip güvenli ve gösterilebilir ilk sürümü tamamlıyoruz.

Yapılacaklar

- [] Production /api/health endpointini test et.
- [] Category, Product ve Customer read endpointlerini test et.
- [] Beş analytics endpointini gerçek production/demo verisiyle test et.
- [] AI query, clarification ve unsupported senaryolarını test et.
- [] AI provider kapalıyken direct analytics'in çalıştığını doğrula.
- [] CORS allowed/disallowed origin testlerini yap.
- [] Production error response içinde stack trace veya secret olmadığını kontrol et.
- [] AI'nın raw SQL çalıştırmadığını ve input length/limit korumalarının aktif olduğunu kontrol et.
- [] Database backup ve rollback yolunu doğrula.
- [] Tüm automated testleri Release configurationda çalıştır.
- [] Dinamik veri demosunu son kez çalıştır.
- [] Açık kritik/yüksek güvenlik bulgusu kalmadığını doğrula.
- [] Release notu oluştur ve çalışan commit'i v1.0.0 olarak etiketle.

Dikkat edilmesi gerekenler

- Testler başarısızken release etiketi oluşturma.
- Authentication henüz yoksa public production riskini gizleme; erişimi sınırla veya README'de açıkça belirt.

Bu adım bittiğinde

- ☐ Must Have kapsamı productionda çalışıyor.
- ☐ Smoke, regression ve güvenlik kontrolleri geçiyor.
- ☐ QueryPilot v1.0.0 gösterime hazırdır.

Bu adımda öğrenilecekler

Smoke test • Security review • Rollback • Release management

6. Nice to Have Backlog

Başlama kuralı

Bu kartlar v1.0.0 Must Have tamamlanmadan başlatılmaz. Her biri bağımsız backlog işidir; ana roadmap'in parçası değildir.

CSV Import Ürün/müşteri verisini şema, boyut ve satır validationı ile içe al.	CSV Export Analytics servisinin mevcut sonuçlarını spreadsheet-safe CSV olarak dışa aktar.
PDF Rapor Seçili analytics sonuçlarını taşmasız kurumsal rapora dönüştür.	JWT ve Roller Admin, Analyst ve Viewer için endpoint yetki politikaları ekle.
Kullanıcı Geçmişi Kullanıcının önceki sorularını PII minimizasyonu ve pagination ile göster.	Çoklu Şirket Her kaydı company/tenant sınırıyla izole et; bu büyük bir mimari değişikliktir.
Otomatik Rapor Zamanlanmış analytics çalıştırma ve güvenli job yönetimi ekle.	Email Raporu Üretilen raporu izinli alıcılara güvenli mail provideri ile gönder.

7. Proje Tamamlandığında

Örnek kullanıcı akışı

Kullanıcı 'Geçen aya göre satışlarımız nasıl?' diye sorar. AI soruyu anlar; backend iki tarih aralığını belirler; PostgreSQL siparişleri sorgulanır; satışlar ve yüzde değişimi backend tarafından hesaplanır; AI gerçek sonucu açıklar; frontend grafik gösterebilir.

Ortaya çıkan ürün

- PostgreSQL üzerinde ilişkisel ürün, müşteri, sipariş ve iade sistemi.
- Feature bazlı sade ASP.NET Core REST API.
- Fiyat snapshotı ve backend total hesabıyla güvenli sipariş akışı.
- Kısmi/tam iade ve over-return koruması.
- Beş temel BI analizi ve previous period comparison.
- Doğal dil soru anlama, clarification, güvenli routing ve grounded açıklama.
- Unit/integration/AI contract testleri, Swagger, CORS, README ve production deployment.
- Database değişimine göre sonucu değişen, hardcode olmadığı kanıtlanmış demo.

CV için Türkçe örnek

- ASP.NET Core ve PostgreSQL ile feature bazlı modüler REST API geliştirdim.
- Ürün, müşteri, sipariş, sipariş kalemi ve iade için ilişkisel veri modeli tasarladım.
- EF Core ve SQL aggregate sorgularıyla dinamik satış, trend, kategori ve iade analizleri uyguladım.
- LLM tabanlı doğal dil katmanını structured output, backend validation ve güvenli intent routing ile entegre ettim.
- AI cevaplarını gerçek database sonuçlarına dayandırdım; automated testler ve cloud deployment ekledim.

English CV example

- Developed a feature-based ASP.NET Core REST API backed by PostgreSQL and EF Core.
- Designed relational models and business rules for products, customers, orders, line items and returns.
- Implemented dynamic sales, trend, product, category and return analytics using database aggregations.
- Integrated an LLM layer with structured intent extraction, backend validation and allow-list routing.
- Grounded AI explanations in real database results and added automated tests and cloud deployment.

İlk hareket

ADIM 1 kartını açın ve checklisti tamamlayın. 'Bu adımı bittiğinde' bölümündeki sonuçlar çalışmadan ADIM 2'ye geçmeyin.